杭州電子科技大学

软件工程课程设计

（2026 届）

题　　目　基于 Qt6 的桌面助手设计与实现

学　　院　计算机学院

专　　业　软件工程

班　　级

学　　号

学生姓名

指导教师

完成日期　2026 年 12 月

诚 信 承 诺

我谨在此承诺：本人所写的课程设计报告《基于 Qt6 的 Windows 桌面助手设计与实现》均系本人独立完成，没有抄袭行为，凡涉及其他作者的观点和材料，均作了注释，若有不实，后果由本人承担。

承诺人（签名）：

2027 年 1 月 10 日

摘　　要

本课程设计完成了一个面向 Windows 平台的桌面助手软件 Assistant。该软件以全局热键（默认 Alt+Space）唤起搜索浮窗为核心交互，同时提供常驻主面板，用于待办事项、记事本、剪贴板历史、插件市场与设置等扩展功能；在其之上规划了文件管理、传输与共享、安全、插件四个可独立演进的功能模块。

软件采用 C++20 与 Qt6 Quick（QML）实现，使用 CMake 构建、vcpkg 管理依赖。文件检索子系统已完整落地：以 Win32 的 FindFirstFileExW 构建句柄栈深度优先扫描器，用不可变列式快照与写时复制方式发布索引，查询侧以四档前缀语义打分配合贪心子序列匹配与有界编辑距离，并通过 top-K 最小堆只保留最优结果；索引与查询全链路零锁读取，扫描在后台线程完成，冷启动后界面先可用、索引随后就绪。插件市场模块实现了插件包解析、市场目录、本地与在线安装、页面贡献与导航注入的完整闭环。

实测表明：在约 3.2 万个真实文件的索引规模下，扫描耗时约 0.46～0.50 秒；在 2 万条语料上，1～3 字符查询的 Release 耗时约 3.9 毫秒，较旧实现在同一语料上提速约 4.8 倍，并在 20 万条语料上取得 7.8 倍加速。系统在 Windows 10/11 上运行稳定，热键、托盘、开机自启动与配置持久化均通过真机验证。

关键词：桌面助手；文件检索；插件系统；Qt6；软件工程

ABSTRACT

This project implements Assistant, a desktop assistant for Windows. A global hotkey (Alt+Space by default) opens a search palette, and a resident main panel hosts todo lists, notes, clipboard history, a plugin market and settings. On top of this core, four independently evolvable modules are planned: file management, transfer and sharing, security, and plugins.

The software is written in C++20 with Qt6 Quick (QML), built by CMake and vcpkg. The file indexing subsystem is complete: a Win32 FindFirstFileExW based scanner performs a handle-stack depth-first walk; an immutable columnar snapshot is published copy-on-write; queries are scored by a four-tier prefix scheme combined with greedy subsequence matching and bounded edit distance, and only the best results are kept in a top-K min-heap. Reads are lock-free along the whole query path and scanning runs on a background thread, so the window becomes usable before the index is ready. The plugin market covers package parsing, a market catalogue, local and online installation, page contribution and navigation injection.

On a real corpus of about 31,845 files, scanning takes 459-502 ms. On 20,000 records a 1-3 character query takes about 3.9 ms in Release builds, 4.8 times faster than the previous implementation, and 7.8 times faster on 200,000 records. The system runs stably on Windows 10 and 11; hotkey, tray, auto-start and configuration persistence were verified on real machines.

Key words: desktop assistant; file indexing; plugin system; Qt6; software design

目　　录

1　引言 1
1.1　项目背景与意义 1
1.2　国内外同类软件现状 1
1.3　主要研究内容 1
1.4　论文组织结构 2
2　需求设计 3
2.1　系统概述 3
2.2　功能需求 3
2.3　非功能需求 3
2.4　主要技术指标 4
2.5　可行性分析 4
3　系统设计 5
3.1　总体架构 5
3.2　模块划分与职责 5
3.3　模块接口与契约 6
3.4　数据设计 6
3.5　关键流程设计 7
3.6　并发与线程设计 8
3.7　异常处理与恢复设计 8
3.8　构建与部署设计 8
4　详细设计 9
4.1　文件索引子系统 9
4.2　查询与打分 10
4.3　主面板与扩展功能 11
4.4　插件市场 11
4.5　设备同步 12
4.6　安全模块 12
5　测试 13
6　实现 14
7　结论 15
致谢 16
参考文献 17
附录 18

1　引言

1.1　项目背景与意义

个人计算机的本地数据规模在过去十年持续膨胀：一个普通开发者的工作目录里往往同时存在代码仓库、构建产物、设计稿、临时下载与多套文档版本；而操作系统的文件管理方式仍以树形目录和关键字命名为中心。用户记得“三天前改过的那份报价单”，却未必记得它叫什么、放在哪个盘。与此同时，桌面端高频操作——打开应用、查剪贴板、记一条待办、算一个式子——被分散在若干互不相通的小工具里。

桌面助手的价值正在于此：用一个全局热键把所有“顺手要用”的能力收拢到一个入口，把检索、启动、记录、传输与扩展做成一套可组合的底座。它既是一个实用的效率工具，也是一个足够真实的软件工程练习——涉及 Windows 原生 API、多线程与并发数据结构、SQLite 持久化、QML 与 C++ 的双向绑定、插件沙箱边界与安全策略等一系列在课堂示例中难以覆盖的工程问题。

1.2　国内外同类软件现状

国外方面，macOS 平台的 Alfred、Spotlight 与 Windows 平台的 PowerToys Run、Wox、Listary 构成了桌面启动器与文件检索的主要参照。它们的共同思路是“键盘优先、模糊匹配、结果按使用频率排序”，差别在于索引策略：Spotlight 依赖系统级元数据索引，Wox 与 PowerToys Run 依赖插件提供的检索源，Listary 则常驻注入资源管理器。国内方面，uTools、Quicker 等产品把“插件即能力”的思路做到极致，通过插件市场形成生态，但其插件多运行在受限的 JavaScript 沙箱中，能力边界与性能上限均受解释器约束。

现有产品也暴露出三类问题：其一，检索结果与真实文件系统脱节，索引陈旧或干脆只搜文件名；其二，插件生态缺乏统一的权限与生命周期约定，插件崩溃会拖垮宿主；其三，文件“管、传、护”三件事被拆到不同软件里，用户需要在多个窗口间搬运路径。本设计试图在一套自洽的架构下同时回答“找得到、出得去、护得住、可扩展”四个问题。

1.3　主要研究内容

本课程设计围绕一个可运行的 Windows 桌面助手，完成以下工作：

（1）实现全局热键唤起的搜索浮窗与常驻主面板双窗口交互，完成托盘、开机自启动与配置持久化；

（2）设计并实现文件索引子系统：原生扫描、三方差量持久化、不可变快照与无锁查询、分级打分与 top-K 截断；

（3）实现主面板扩展功能：待办、记事本、剪贴板历史与设置（主题、日志级别、热键自定义）；

（4）设计四模块（文件管理、传输与共享、安全、插件）的总体架构、模块契约与依赖

规则；

（5）实现插件市场的完整闭环：插件包与清单、市场目录、安装流水线、页面贡献与导航注入；

（6）完成关键路径的性能实测与对比，形成可复现的验收基线。

1.4 论文组织结构

全文共分七章。第 1 章介绍背景、现状与研究内容；第 2 章给出需求设计，包括功能需求、非功能需求、技术指标与可行性分析；第 3 章给出系统设计，包括总体架构、模块划分、契约、数据、流程、并发与部署设计；第 4 章为详细设计，按“已完成 / 未完成”如实划分，仅对已落地的子系统展开；第 5 章测试、第 6 章实现、第 7 章结论以及致谢、参考文献与附录，待相应工作完成后补齐。

2　需求设计

2.1　系统概述

Assistant 是一个常驻 Windows 桌面的效率工具。用户按下全局热键后，屏幕中央弹出搜索浮窗，输入即搜；浮窗同时承担计算器、应用启动与文件检索三类结果。主面板则以独立窗口常驻，提供待办、记事、剪贴板、插件市场与设置五个页面。系统的长期目标是把“本地文件”作为一等公民：既能被快速检索，也能被安全地发送、接收与保护，并允许第三方以插件形式扩展能力。

从使用者视角看，系统面向两类角色：普通用户（只使用搜索、待办、剪贴板等基础能力）与扩展开发者（编写并分发 QML 插件）。从系统视角看，全部能力被组织为四个模块：M1 文件管理、M2 传输与共享、M3 安全、M4 插件，四者共享一套文件身份与契约，由应用层统一装配。

2.2　功能需求

系统功能需求按“已交付”与“规划中”两类列出，如表 2-1 所示。其中“已交付”指当前主分支可运行、可验收的功能；“规划中”指已完成设计但尚未落地的部分，后文详细设计不再展开。

表 2-1　功能需求清单

编号	功能	说明	状态
F1	全局热键唤起	Alt+Space 唤起/隐藏搜索浮窗，可在设置页重新绑定	已交付
F2	搜索与启动	应用、命令、计算结果、文件四类结果统一排序	已交付
F3	文件索引	后台扫描用户目录，三方差量写库，快照发布后可查询	已交付
F4	主面板	待办 / 记事本 / 剪贴板 / 插件市场 / 设置五页	已交付
F5	剪贴板历史	轮询系统剪贴板，去重入库，支持搜索与回填	已交付
F6	插件市场	插件安装、启用、卸载，贡献页面进入左侧导航	已交付（分支）
F7	设备同步	以共享目录为通道的文件差量同步与冲突副本	部分完成
F8	安全中心	受控操作的策略判定、完整性账本与保险箱	未开始
F9	传输内核	分块校验、断点续传、接收隔离区与恢复	未开始

2.3　非功能需求

除功能之外，系统还需满足若干工程约束，这些约束直接决定了架构取舍：

表 2-2　非功能需求

类别	需求	验收口径
性能	扫描期间界面不卡顿	扫描在后台线程，主线程无阻塞调用；窗口先于索引可用
性能	2 万条语料的 1～3 字符查询 < 10 ms（Release）	基准程序 assistant_bench 输出 P50/P95
容量	索引硬上限 100 万条，默认不截断真实用户目录	真机索引条目数与磁盘目录数一致
可靠性	异常退出后索引库不损坏，可重扫重建	强杀进程后重启，索引仍可查询
安全	策略异常、超时、审批失效一律拒绝	策略桩注入异常，验证返回 Deny
安全	插件默认进程内完全信任，不做沙箱承诺	文档与界面明确表述，不误导用户
可维护性	模块间不得依赖实现头，只依赖门面与契约	代码评审 + 依赖方向检查
可扩展性	新增页面/插件不修改既有页面代码	导航以 key 组织，插件页运行时注入
可移植性	构建脚本不硬编码第三方库路径	CMake 使用 VCPKG_INSTALLED_DIR

2.4　主要技术指标

表 2-3　主要技术指标与实测值

指标	目标值	实测值（Release）
全量扫描（3.2 万文件）	< 1 s	459～502 ms
目录级剪枝比例	—	97.7%
查询 1～3 字符（2 万条）	< 10 ms	3.9 ms（旧实现 18.7 ms）
查询 1～3 字符（20 万条）	< 50 ms	35.4 ms（旧实现 275 ms）
冷启动到可查询	< 150 ms	28.7 ms
快照内存（20 万条）	< 60 MB	64.4 MB（待优化）
安装包体积	< 100 MB	待打包

2.5　可行性分析

技术可行性方面，所选技术栈均为成熟方案：Qt6 提供跨平台窗口、QML 与 SQL 能力；Win32 的目录遍历与热键注册接口稳定；SQLite 以单文件形式嵌入，无需额外服务。项目已在 Windows 10/11 + VS2022 + vcpkg 环境下完成多轮真机验证，索引、查询、热键、剪贴板等关键路径均有可复现的验收程序。

经济可行性方面，全部依赖开源且免费，开发与运行不需要服务器与带宽投入；插件分发在首版使用静态索引文件，后续再引入远端仓库即可。操作可行性方面，交互遵循“一个热键 + 一个面板”的常见心智模型，学习成本低；安装包为绿色解压形态，不写系统目录。

风险方面，主要不确定性集中在三处：其一，LAN 传输依赖经过验证的 TLS 后端，若目标环境不可用则暂停该功能发布，不降级为自研加密；其二，进程内插件无法提供真正的隔离，需在文档与界面中如实说明；其三，全量扫描在大容量磁盘上仍有 IO 瓶颈，需以增量监听与冷启动预热缓解。

3　系统设计

3.1　总体架构

系统采用四层结构：接入层、模块层、契约层与基座层。依赖方向自上而下，任何反向依赖都视为缺陷。接入层由若干窗口与托盘、热键组成，只通过桥对象调用模块门面；模块层由四个薄门面构成，每个门面只做“持有依赖、转发调用、暴露状态”三件事；契约层是跨模块唯一允许共享的类型集合；基座层提供文件身份、快照、扫描、数据库访问、迁移、配置与日志等公共能力。

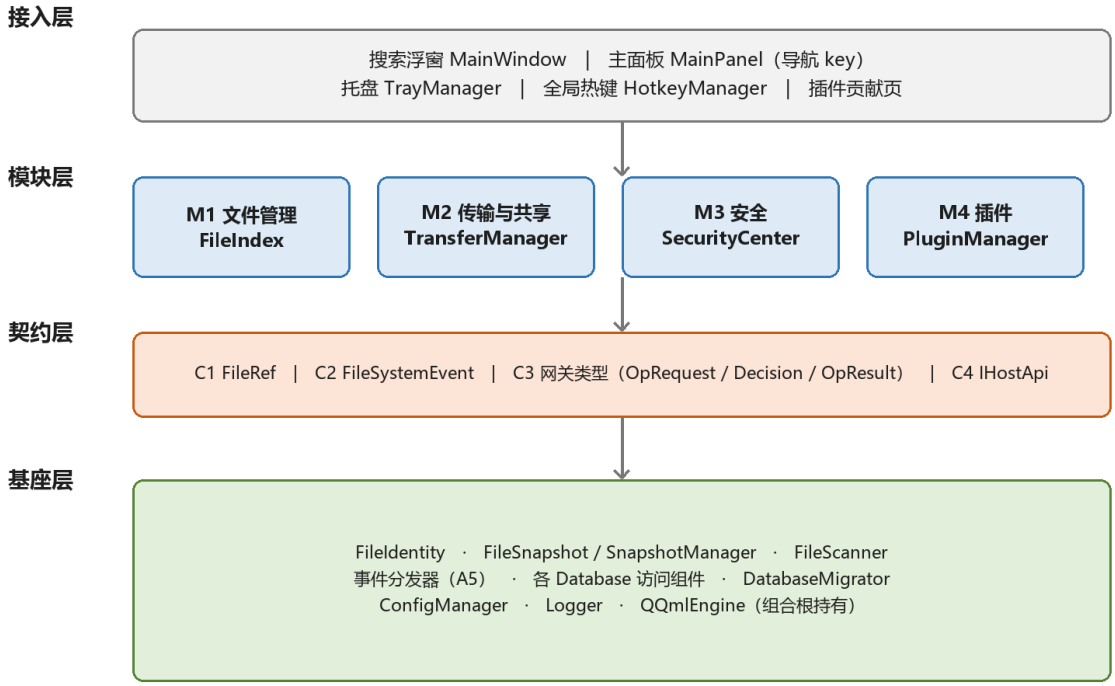


图 3-1　系统分层架构

组合根只有一处：main.cpp。它按固定顺序完成配置加载、日志初始化、数据库迁移、门面构造与依赖注入、QML 引擎加载与异步任务启动。系统中不存在服务定位器，也不存在第二个全局单例（Logger 除外）。

3.2　模块划分与职责

四个模块各自回答一个问题，问题的答案不允许重叠：M1 回答“盘上有什么、在哪、怎么找到”；M2 回答“文件怎么出去、怎么进来”；M3 回答“这件事该不该允许、东西有没有被动过、怎么保护”；M4 回答“谁来加新能力、加的能力能碰什么”。

表 3-1　模块职责划分

模块	门面	主要职责	明确不做
M1 文件管理	FileIndex	索引、查询、分类与排除、box 登记与去向	不判断该不该操作，不负责传输
M2 传输与共享	TransferManager	作业状态机、分块与校验、通道、接收隔离区	不判断文件是什么，不做安全判定
M3 安全	SecurityCenter	策略判定、审批、完整性账本、保险箱	不负责文件定位，不实现传输
M4 插件	PluginManager	插件生命周期、能力声明、页面贡献	不实现以上任何能力本身

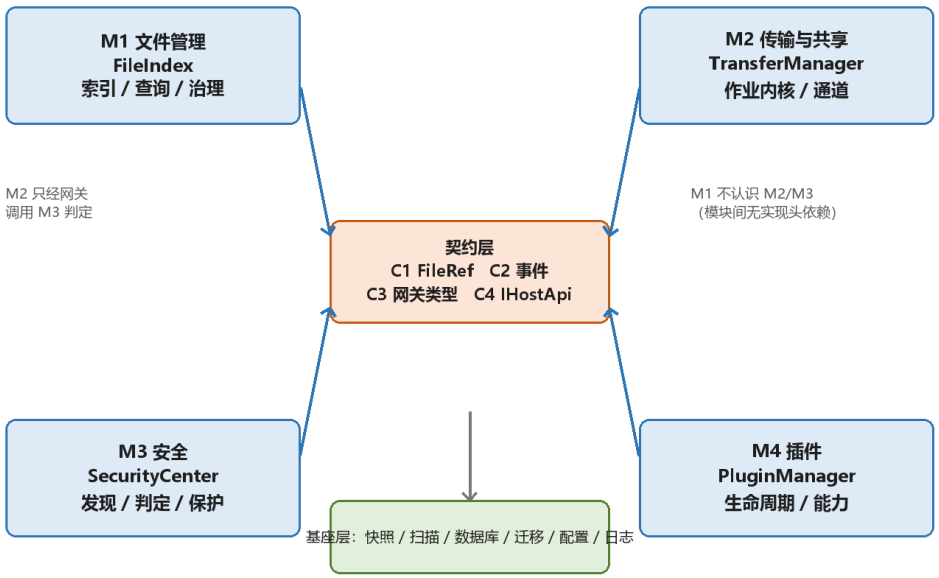


图 3-2　四模块与契约、基座的关系

3.3　模块接口与契约

模块之间只允许三种交互：调用门面上被批准的方法、传递契约层类型、由应用层编排。契约层包含四个类型：C1 FileRef（跨模块轻量文件句柄）、C2 FileSystemEvent（文件系统变化通知）、C3 网关类型（OpRequest / Decision / OpResult）、C4 IHostApi（插件唯一可见的宿主能力）。

表 3-2　跨模块契约

契约	内容	关键不变量
C1 FileRef	id、path、size、mtime、catMask、flags、secLevel	不是授权凭证；受控操作前必须重新解析
C2 FileSystemEvent	kind、目录、文件名、generation、overflow	可重复可丢失；溢出必须触发复核
C3 网关类型	请求主体、操作类型、判定结果、执行结果	仅 Allow/Ask/Deny；异常一律 Deny
C4 IHostApi	数据目录、设置、通知、页面贡献、能力枚举	能力名固定枚举；宿主解析调用主体

3.4　数据设计

系统采用“每个功能一个数据库文件、一个独立连接名、一个唯一写者”的策略。SQLite 以 WAL 模式运行，连接与线程绑定，跨线程只传递值而不共享连接。数据库按权威性分为三类：权威库（丢失不可重建）、半权威库（作业可弃、配对不可弃）与派生库（可随时重扫重建）。

表 3-3　数据库清单与迁移策略

数据库	性质	唯一写者	迁移失败策略
history.db	权威	HistoryDatabase	禁用历史功能
todo.db / notes.db / clipboard.db	权威	各自 Database	禁用对应功能
file_index.db	派生	FileIndexDatabase	备份后重建（重扫）
sync.db	半权威	SyncDatabase	降级启动，禁用传输
security.db（规划）	权威	SecurityDatabase	停止相关功能并拒绝启动

除数据库外，系统还维护三类非结构化数据：不可变文件快照（内存）、JSON 配置文件（含主题、热键、索引引擎开关等）以及按大小滚动的日志文件。配置读取失败时回退到内置默认值并立即回写，避免“配置损坏导致启动即崩”。

3.5　关键流程设计

（1）搜索流程。用户输入经 40 ms 防抖后进入 AppBridge，门面取一份当前快照（shared_ptr 拷贝，不加锁），查询解析器把输入拆成自由词与否定词，打分内核逐条计算分数，查询引擎以最小堆维护 top-K，最后由桥层组装成界面模型。图 3-3 给出索引与查询两条数据流。

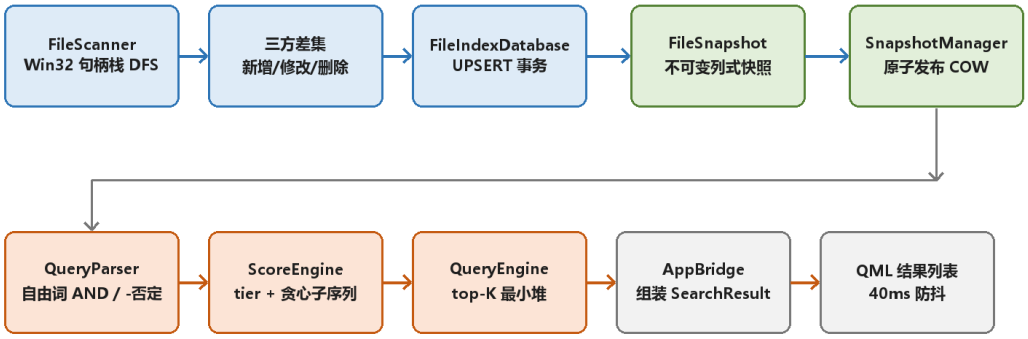


图 3-3　索引构建与查询数据流

（2）受控操作流程。删除、移动、外发等操作必须经网关：校验调用主体、重新解析目标对象、调用策略引擎得到判定、必要时等待用户审批并在审批后再次校验、由受控执行器完成实际 I/O、记录结果并发出带操作号的事件，消费者据此去重。该流程保证“策略异常不默认放行”“检查与使用之间不留竞态”。

（3）接收文件流程。远端字节先落到隔离区，经分块哈希与最终内容哈希校验、路径安全校验后，以原子重命名落到目标目录（默认不覆盖同名文件），再登记来源并更新索引；任一步失败都停留在隔离区或安全清理，不允许出现“半入库”。

（4）插件安装流程。安装器以状态机推进“检查—下载—解压—校验—提交”五个阶段，全过程在失败时清理临时目录；提交成功后由注册表落盘、加载器创建页面对象并通知导航刷新。



图 3-4　插件安装流水线

3.6　并发与线程设计

系统中有三类线程：GUI 线程负责界面、桥对象与插件 QML 的执行；索引线程负责扫描、差量写库与快照构建；同步线程负责一次同步作业（含数据库访问）。三者之间只通过只读快照、值语义信号与队列连接通信。写侧永远构建新对象再原子发布，读侧拿到 shared_ptr 后即使写侧继续发布也不会观察到中间状态，这是查询路径能够完全无锁的根本原因。

线程约束被写成硬规则：一个 QSqlDatabase 连接只在一个线程内使用；工作线程不得直接触碰任何 QML 对象；长耗时任务必须可取消，取消点放在目录遍历与文件分块的边界上。

3.7　异常处理与恢复设计

系统级失败按“可重试、可补偿、需人工”三类处理。索引库损坏属于可重建：删除派生库后重扫即可恢复；配置损坏属于可回退：使用默认值并记录告警；跨库流程（接收、入库、审计）属于可补偿：以持久化状态机记录每一步，重启后从断点继续或安全清理。安全相关的失败一律向“拒绝”收敛，不允许出现“判定失败但放行”的分支。

3.8　构建与部署设计

构建使用 CMake 与 vcpkg，编译选项为 C++20 / MSVC x64。Qt 的部署采用两层机制：构建后自动生成 qt.conf 指向 vcpkg 的 Qt 插件与 QML 目录，同时生成 run.bat 把 Qt 的 bin 目录加入 PATH，二者共同解决 Windows 下插件与 DLL 的查找问题。部署脚本不硬编码第三方库路径，改由 VCPKG_INSTALLED_DIR 与 VCPKG_TARGET_TRIPLET 推导，换机器或换 vcpkg 位置无需改脚本。

4　详细设计

本章只展开已经完成并经过真机验收的部分。尚未完成的功能仅保留小节标题，待实现后补充。

4.1　文件索引子系统

4.1.1　模块结构

文件索引子系统由六个部分组成：门面 FileIndex、采集器 FileScanner、内存表示 FileSnapshot、发布器 SnapshotManager、查询引擎 QueryEngine（内部含 QueryParser 与 ScoreEngine）、持久化组件 FileIndexDatabase 与迁移器 DatabaseMigrator。门面只持有依赖、转发调用并暴露统计信息，不实现遍历、SQL 或打分逻辑。

这种拆分解决了一个具体问题：索引重建期间界面不能卡。若把扫描、写库、查询放在同一对象里，锁的粒度必然退化到“整个索引”，而不可变快照把“写”和“读”彻底分开——写侧构建新快照，读侧永远读旧快照，发布只是一次指针替换。

4.1.2　采集层：Win32 句柄栈深度优先遍历

采集层没有使用 QDirIterator，原因是它的过滤器只决定“返回哪些条目”，不决定“是否递归”，而 Windows 的目录联结（junction）在 Qt 中既不是文件、isSymLink() 也返回假，会导致递归进入循环。实现改为自行维护句柄栈：进入目录时打开一次句柄，判定为 reparse point 或命中剪枝名单就整棵子树跳过，否则压栈继续。遍历顺序与递归前序逐条相同，这一点并非洁癖——索引有硬上限，截断按顺序进行，顺序变了截出来的集合也就变了，验收对比会随之失真。

为提升吞吐，扫描使用 FindFirstFileExW 配合 FindExInfoBasic 与 FIND_FIRST_EX_LARGE_FETCH，一次系统调用批量返回目录项（含大小与时间），避免逐条 stat；长路径统一加 \\?\ 前缀；权限拒绝的目录计入统计而不中断整轮扫描。实测在 3.2 万文件的真实语料上，全量扫描耗时 459～502 ms，目录级剪枝跳过了 97.7% 的候选目录。

4.1.3　持久化与三方差量

file_index.db 由 dirs、files、roots 三张表组成，属于派生数据：删库重扫即可重建，因此迁移失败时选择“备份后重建”而不是“拒绝启动”。每次扫描结束后，系统把“磁盘上的条目集合”“数据库中的条目集合”与“内存快照中的条目集合”三者做差集，得到新增、修改与删除三类变更，在一个事务内批量 UPSERT 与删除。差集收敛性以验收程序验证：连续两轮扫描后，变更集合为空。

数据库迁移使用 PRAGMA user_version 记录版本，逐版本前向迁移，每步一个事务，禁止“删表重建”。迁移器在打开数据库后立即执行，早于任何业务读写，因此不会出现“代码已升级但表结构未升级”的中间态。

4.1.4　快照构建与发布

FileSnapshot 采用列式存储：文件名、小写名、目录、大小、时间分别放在独立数组中，查询时按索引访问，避免逐条对象分配。构建由 Builder 完成，构建结束后对象即冻结，外部只能通过只读访问器读取。SnapshotManager 以 std::atomic<std::shared_ptr<const FileSnapshot>> 完成发布：写侧一次 store，读侧一次 load 即得到一份稳定引用，既不需要读写锁，也不会读到半成品。

门面在扫描完成后于 GUI 线程发出 indexReady 信号，携带条目数与耗时；界面据此提示“索引就绪”。扫描期间再次触发扫描不会并发执行，而是记录“待重扫”标记，等当前轮结束后再补一轮，避免多轮扫描互相覆盖差量结果。

4.2　查询与打分

4.2.1　查询解析

查询语法在本阶段保持最小：按空白切分自由词，词之间取 AND；以减号开头的词表示否定。解析器对整串只做一次小写化，后续打分复用同一份小写串，避免逐条记录重复转换。解析结果是一个轻量结构，不含任何堆分配，便于在查询路径上零成本传递。

4.2.2　打分内核

打分采用四档语义：完全相等、前缀命中、词边界命中、子串包含，分别对应固定的分值区间；四档都不命中时，再用“贪心子序列”判断是否为缩写式匹配（如用 “vs” 命中 “Visual Studio”），命中则落在专门的中间档，避免与真正的子串匹配混淆。最后一档是有界编辑距离，仅对已经通过前几档筛选的少量候选计算，且编辑距离上限随词长自适应。

表 4-1　分级打分档位

档位	判定	分值区间	说明
1 精确	整串相等（忽略大小写）	1000	最高优先，直接置顶
2 前缀	以查询串开头	900～999	按剩余长度递减
3 词边界	命中单词首字母位置	800～899	多词查询时逐词累计
4 子序列	贪心匹配查询字符顺序	351～399	缩写式匹配，零分配
5 包含	子串出现	200～350	按出现位置递减
6 近似	有界编辑距离内	0～199	仅对前序候选计算

打分内核的另一条约束是“零分配”：整个逐条打分过程中不构造 QString、不申请堆内存，所有中间状态放在栈上或复用缓冲区。这条约束带来的是可预测的延迟——查询耗时不随 GC 或分配器抖动。

4.2.3　结果组装与性能

查询引擎不返回全部命中，而是维护一个容量为 K 的最小堆：遇到比堆顶更高的分数就替换堆顶，最终堆内即为 top-K。相比“全部打分后排序”，该策略把复杂度从 O(n log n) 降到 O(n log K)，在 K 远小于 n 的场景下收益明显。

门面把结果转发给桥层，由桥层补齐展示字段（标题、副标题、图标、类型）并转成界面模型。基准程序以固定语料重复查询并输出 P50/P95：2 万条语料上 1～3 字符查询为 3.9 ms，较旧实现（18.7 ms）提速 4.8 倍；20 万条语料上为 35.4 ms，较旧实现（275 ms）提速

7.8 倍。需要说明的是，Debug 构建同口径为 114 ms，所有性能结论均以 Release 为准。

4.3 主面板与扩展功能

4.3.1 待办、记事本与剪贴板

三个功能各自使用独立数据库与独立桥接接口，互不共享连接。待办支持“记录—完成—划去”的状态流转，完成项保留但不参与默认排序；记事本提供列表与编辑两态，编辑内容以覆盖写方式落库；剪贴板以固定间隔轮询系统剪贴板，对文本内容做 SHA-256 去重后入库，界面按相对时间展示，每分钟自刷新一次，悬停显示原始时间戳。

三个页面的列表模型都由桥对象以 QVariantList 形式提供，增删改后发出对应信号，QML 侧只做渲染，不持有业务状态。这样做的直接好处是：页面切换不会丢失数据，进程重启后状态完全由数据库决定。

4.3.2 设置页与系统集成

设置页承载四类配置：主题（浅色/深色）、日志级别、全局热键与开机自启动。热键修改的流程是“捕获组合键 → 调用重注册 → 失败自动回滚原热键”，避免用户改成一个被占用的组合键后再也唤不出窗口；开机自启动以注册表 HKCU\Software\Microsoft\Windows\CurrentVersion\Run 为唯一真相来源，不写入配置文件，避免“配置说开了、注册表没了”的不一致。

主题色抽为 QML 单例 Style.qml，六个页面的重复色值收敛到一处；版本号由 CMake 注入并显示在关于区域。托盘菜单提供显示/隐藏、打开主面板与退出三个动作，与热键共享同一套窗口显示逻辑。

4.4 插件市场

插件市场在独立分支上完成，尚未合并到主分支，本节描述其已实现的设计。

4.4.1 插件包与清单

插件以 zip 包分发，包内必须包含 plugin.json 清单与入口 QML 文件。清单声明标识、名称、版本、作者、入口文件、宿主版本区间、权限列表与三类贡献点：页面、检索提供者与命令。清单解析采取“全有或全无”策略：任何字段不合法都拒绝安装，避免半合法清单流入后续环节造成难以定位的问题。首版只支持 QML 运行时，字段中预留的 native 运行时会被解析但在安装阶段拒绝。

4.4.2 安装流水线

安装器是一个显式状态机，依次执行检查、下载、解压、校验与提交。解压阶段逐条校验目标路径，拒绝跳出插件目录的相对路径（zip-slip）；校验阶段核对清单字段与包内文件哈希；提交阶段以原子重命名把临时目录转为正式安装目录，失败时清理临时目录，不留半安装状态。市场目录支持三种来源：内置索引、本地缓存与远端地址，远端不可达时自动回退到缓存并标注“离线”。

4.4.3 加载与宿主接口

加载器为每个插件创建独立的 QML 上下文，并向其中注入唯一一个宿主对象 PluginHost。该对象只暴露只读属性（插件标识、版本、数据目录、主题）与少量动作（提示、读写自己的设置），不暴露引擎、注册表或管理器本身。插件的数据只能写入自己的数据目录，安装目录按约定只读——升级时整目录替换，写入其中的数据必然丢失。

卸载流程强调顺序：先销毁页面对象与宿主对象，再删除安装目录，最后更新注册表；初始化失败的插件会被记录为“问题插件”，不阻塞其余插件的加载。

4.4.4 导航注入

主面板左侧导航从“索引对齐”改为“键对齐”：内置页面与插件页面都以 key 标识，页面内容由 key 计算得出，插件页由 C++ 侧创建并交给加载器托管。这一改动使“增加一个页面”不再影响任何既有页面的索引判断，也是插件能够贡献页面的前提。

4.5 设备同步

4.6 安全模块

5　测试

6　实现

7　结论

致谢

参考文献

附录